Capítulo 3: Compute Shaders y Jerarquía de Memoria

Memoria compartida on-chip (`var<workgroup>`), barreras y algoritmos paralelos avanzados

Serie: *BrowserOS: Computación Nativa en la Web con Wasm, WebGPU y WGSL* | **Autoría:** Antigravity Engineering Lab
Referencias en O'Reilly: Programming Massively Parallel Processors (4th Ed.), Computer Organization and Design y GPU-Accelerated Computing with Python 3 and CUDA

1. La Pared de Memoria (The Memory Wall) y el Modelo Roofline

En el diseño de sistemas de computación gráfica y aceleración de IA, el cuello de botella físico primario no es la velocidad matemática de las ALUs, sino el suministro de datos. Mientras que una GPU moderna ofrece decenas de TFLOPs de cómputo, sus procesadores deben lidiar con una penalización masiva cuando leen operandos desde los módulos de memoria externa (VRAM / DRAM): una latencia de **200 a 800 ciclos de reloj**. Este estrangulamiento físico se define formalmente como la **Pared de Memoria (The Memory Wall)**.

Como formulan Patterson & Hennessy (**Computer Organization and Design**), la GPU oculta parte de esta latencia alternando rápidamente entre miles de hilos concurrentes (**latency hiding**). No obstante, si el algoritmo no reutiliza los datos en el silicio, el bus de DRAM se colapsa.

Wen-mei W. Hwu, David B. Kirk e Izzat El Hajj (**Programming Massively Parallel Processors**) formalizan este principio mediante el **Modelo del Techo (Roofline Model)** y la **Intensidad Aritmética** (I_A), que mide las operaciones en coma flotante (FLOPs) computadas por cada byte transferido desde la DRAM:

$$I_A = \frac{\text{Operaciones de Cómputo (FLOPs)}}{\text{Tráfico de Memoria Global (Bytes)}}$$

Para que un algoritmo escape del régimen limitado por memoria (**Memory-Bound**) y alcance el techo máximo de cómputo del chip (**Compute-Bound**), es imperativo almacenar y reutilizar datos en la memoria de alta velocidad integrada en el propio procesador: la memoria compartida (`var<workgroup>`).

2. Jerarquía Física de Memoria en WebGPU

WGSL modela directamente la física multinivel del procesador gráfico:

Nivel de Memoria	Declaración WGSL	Ámbito (Scope)	Latencia	Características Físicas en Silicio
Registros Privados	`let`, `var<private>`	Hilo individual	0 - 1 ciclos	Integrado en las ALUs. Ancho de banda > 20 TB/s.
Memoria Compartida	**`var<workgroup>`**	**Workgroup (Bloque)**	**1 - 10 ciclos**	**SRAM On-Chip (16 KB a 64 KB). Ancho de banda 15 TB/s.**
Caché Uniforme	`var<uniform>`	Global (Broadcast)	5 - 15 ciclos	Lectura solo-lectura óptima con caché L1/L2.
Memoria Global	`var<storage>`	Universal (GPU/CPU)	200 - 800 ciclos	VRAM externa (GDDR6/HBM). Capacidad de Gigabytes.

Límites de Silicio en WebGPU: El estándar garantiza un mínimo universal de **16.384 bytes (16 KB)** por workgroup (`maxComputeWorkgroupStorageSize`), lo que permite reservar con seguridad 4.096 floats (`f32`) compartidos entre los hilos del grupo.

3. Bancos de Memoria y Conflictos de Acceso (Bank Conflicts)

La memoria compartida en chip está organizada en **32 bancos de memoria independientes** de 4 bytes (32 bits).

- **Acceso Óptimo (1 ciclo):** Si los 32 hilos de un warp acceden simultáneamente a palabras en bancos diferentes, o leen la misma dirección (**Broadcast**), la operación se resuelve en un único ciclo de reloj.
- **Conflicto de Bancos (Bank Conflict):** Si dos o más hilos intentan leer o escribir direcciones distintas mapeadas en el mismo banco, las transferencias se serializan forzosamente, degradando el rendimiento de la SRAM on-chip en un factor proporcional al número de colisiones.

4. Primitivas de Sincronización: `workgroupBarrier()` y `storageBarrier()`

La concurrencia masiva exige evitar carreras de datos (**RAW, WAR, WAW**):

1. `workgroupBarrier()`: Detiene la ejecución de todos los hilos del workgroup hasta que todos hayan alcanzado este punto y garantiza que todas las escrituras previas en `var<workgroup>` sean visibles de inmediato para todos los hilos del bloque.
2. `storageBarrier()`: Sincroniza la visibilidad de memoria para búferes de almacenamiento global (`var<storage, read_write>`) entre los hilos del mismo workgroup.

Regla Sagrada contra la Divergencia de Barreras: Jamás invoques una barrera dentro de una rama condicional que no tomen todos los hilos del grupo (ej. `if (tid < 10) { workgroupBarrier(); }`). Si un subconjunto de hilos se suspende a la espera de compañeros que jamás alcanzarán la barrera, la GPU entra en **bloqueo permanente (Deadlock)** y el sistema operativo reinicia el controlador gráfico (**device lost**). Toda barrera debe situarse en **flujo de control uniforme**.

5. Algoritmo 1: Reducción Paralela en Árbol ($O(\log N)$)

Para sumar miles de elementos sin incurrir en la lentitud lineal $O(N)$ de una CPU, organizamos los hilos en un árbol binario logarítmico:

- **Carga inicial colaborativa:** Cada hilo copia un elemento desde DRAM al array en memoria compartida `s_data` y espera en `workgroupBarrier()`.
- **Plegado sin Divergencia SIMD:** En lugar de usar el operador módulo (`tid % 2 == 0`), se utiliza un paso descendente (`stride >>= 1`) con condición de índice compacto (`if (tid < stride)`). De este modo, los hilos activos ocupan siempre posiciones contiguas (0 a `stride - 1`), manteniendo a warps completos trabajando al 100% de ocupación y warps inactivos totalmente apagados sin divergencia interna.

6. Algoritmo 2: Multiplicación Matricial por Bloques (Tiled GEMM)

La multiplicación matricial ($C = A \times B$) es la piedra angular del aprendizaje profundo en el navegador:

- **El Desastre Ingenuo:** Leer directamente de DRAM para cada producto punto requiere $2N^3$ accesos lentos. Para $N = 4096$, supone transferir más de 500 GB de datos por la memoria externa ($I_A = 0.25$ FLOP/Byte), colapsando el bus de VRAM.
- **La Optimización por Baldosas (Tiling):** Dividimos las matrices en sub-bloques de 16×16 floats cargados cooperativamente en `tile_a` y `tile_b` (`var<workgroup>`). Tras la sincronización con `workgroupBarrier()`, cada dato cargado es reutilizado **16 veces consecutivas** desde la SRAM ultrarrápida (4 ciclos de latencia).
- **Resultado Físico:** Reducción del tráfico a memoria global en un factor de 16x (ahorro del **93.75% de ancho de banda**), elevando la intensidad aritmética a **32.0 FLOP/Byte** y desatando el rendimiento nativo del silicio.

7. Taller Práctico: Laboratorio de Silicio y Memoria en Vivo

Se ha implementado el visualizador interactivo **`workshop_06_tiled_gemm_and_reduction.html`**:

- Alternancia en vivo entre **Tiled GEMM** y **Reducción en Árbol**.
- Simulador visual de baldosas que ilumina en tiempo real la transferencia de celdas desde la DRAM global hasta la SRAM on-chip a lo largo de la dimensión K.
- Diagrama interactivo del plegado de 16 hilos en reducción demostrando la ausencia de divergencia SIMD.
- Panel de telemetría de hardware con cálculo de intensidad aritmética y comparación directa de latencias (DRAM 600 ciclos vs SRAM 4 ciclos).

8. Bibliografía y Lecturas Recomendadas para Profundizar

Obra y Autoría (O'Reilly)	Aportación Clave al Dominio de Memoria y Algoritmos Paralelos
Programming Massively Parallel Processors (4th Ed.) W. Hwu, D. Kirk, I. El Hajj (Morgan Kaufmann)	El tratado canónico sobre computación masiva. Sus capítulos sobre memoria compartida, descomposición por baldosas (*tiling*), resolución de conflictos de bancos y reducciones en árbol constituyen la base teórica y matemática de este capítulo.
Computer Organization & Design D. Patterson & J. Hennessy (Morgan Kaufmann)	La referencia indiscutible sobre arquitectura de procesadores. El análisis de la Pared de Memoria (*Memory Wall*) y el modelo Roofline desmitifica las restricciones físicas de latencia y ancho de banda en procesadores paralelos.
GPU-Accelerated Computing with Python 3 and CUDA N. Cautaerts & H. Ghorbanfekr (Packt Publishing)	Guía contemporánea de optimización práctica. Detalla la estructura física de los 32 bancos de SRAM en chip, el acceso coalescente y la eliminación de carreras de datos mediante barreras de sincronización.
Advances in GPU Research and Practice Hamid Sarbazi-Azad (Morgan Kaufmann)	Compendio de investigación avanzada sobre interconexiones en chip, redes de comunicación de baja latencia y consistencia de memoria en arquitecturas heterogéneas modernas.

Obra y Autoría (O'Reilly)	Aportación Clave al Dominio de Memoria y Algoritmos Paralelos
Vulkan™ Programming Guide Graham Sellers (Addison-Wesley)	La guía técnica oficial del estándar gráfico de bajo nivel sobre el que se implementa WebGPU. Esencial para comprender las barreras de ejecución y visibilidad a nivel de controlador nativo.

9. Próximo Paso: Hacia el Capítulo 4

En el **Capítulo 4: Texturas, Samplers y Computación Gráfica Avanzada en WGSL**, exploraremos el almacenamiento espacial en curva Z (*Morton order*), unidades físicas de filtrado e interpolación bilineal/trilineal gratuita por hardware, y la implementación de kernels de convolución bidimensional en compute shaders.